

Oracle PL/SQL – Fondamenti e Prime Operazioni

Prima lezione del percorso PL/SQL per sviluppatori e analisti con basi SQL. L'obiettivo è introdurre il linguaggio procedurale di Oracle e scrivere i primi blocchi di codice funzionanti.

LIVELLO BASE

ORACLE DATABASE

PREREQUISITO: SQL



SQL vs PL/SQL: Due Linguaggi, Un Ecosistema

Prima di scrivere codice PL/SQL, è fondamentale capire come si posiziona rispetto a SQL. I due linguaggi sono complementari, ma hanno nature profondamente diverse.

SQL – Linguaggio Dichiarativo

SQL descrive **cosa** si vuole ottenere, non come ottenerlo.

- Ottimizzato per l'accesso ai dati
- Esegue operazioni su insiemi (set-based)
- Nessuna logica di controllo del flusso
- Esempio: `SELECT name FROM employees WHERE dept = 10`

PL/SQL – Linguaggio Procedurale

PL/SQL descrive **come** eseguire un'operazione, passo per passo.

- Aggiunge logica, condizioni e cicli
- Gestisce errori e transazioni
- Elabora i dati riga per riga quando necessario
- Esempio: `IF salary > 50000 THEN ... END IF;`

 PL/SQL incorpora SQL al suo interno: puoi usare qualsiasi statement SQL dentro un blocco PL/SQL.

Cos'è PL/SQL?

PL/SQL (Procedural Language / SQL) è l'estensione procedurale del linguaggio SQL sviluppata da Oracle Corporation. Introdotto nei primi anni '90, è oggi il linguaggio di programmazione nativo del database Oracle.

Estensione di SQL

Non sostituisce SQL, lo **potenzia**. Ogni statement SQL valido può essere inserito in un blocco PL/SQL senza modifiche.

Logica e Controllo

Introduce strutture di controllo del flusso: **IF/THEN/ELSE**, cicli **LOOP**, **WHILE**, **FOR** e gestione degli errori.

Esecuzione Server-Side

Il codice viene eseguito **direttamente sul server Oracle**, riducendo il traffico di rete e migliorando le prestazioni.

Usi Principali

Stored procedures, funzioni, trigger, package, job schedulati e blocchi anonimi eseguiti ad hoc.



Blocchi e Blocchi Anonimi

L'unità fondamentale di programmazione in PL/SQL è il **blocco**. Esistono due categorie principali: i blocchi denominati (stored procedure, funzioni, package) e i **blocchi anonimi**, che sono il punto di partenza per imparare PL/SQL.

Blocchi Denominati

Oggetti persistenti memorizzati nel dizionario del database:

- **Stored Procedure** – eseguono azioni
- **Funzioni** – restituiscono un valore
- **Package** – raggruppano procedure e funzioni
- **Trigger** – si attivano su eventi DDL/DML

Rimangono nel database fino a cancellazione esplicita.

Blocchi Anonimi

Blocchi **senza nome**, non memorizzati nel database. Vengono scritti ed eseguiti al volo, tipicamente in strumenti come SQL Developer o SQL*Plus.

- Eseguiti **una sola volta**
- Ideali per test, script e operazioni one-shot
- Base di apprendimento del PL/SQL
- Terminati con `/` sulla riga successiva

📌 Tutti i concetti appresi sui blocchi anonimi si applicano direttamente agli oggetti denominati.

Struttura del Blocco PL/SQL

Ogni blocco PL/SQL segue una struttura standard composta da quattro sezioni. Solo la sezione `BEGIN . . . END` è obbligatoria; le altre sono opzionali ma fortemente consigliate.



Il diagramma illustra le quattro sezioni fondamentali di un blocco PL/SQL nell'ordine in cui vengono elaborate: dalla dichiarazione delle variabili, all'esecuzione della logica, alla gestione degli errori, fino alla chiusura del blocco.

DECLARE – Sezione Opzionale

Si dichiarano variabili, costanti, cursori e tipi personalizzati. Tutto ciò che viene usato nel blocco deve essere dichiarato qui, prima dell'esecuzione.

- Variabili locali con tipo e valore iniziale
- Cursori espliciti per query multi-riga
- Tipi definiti dall'utente

BEGIN – Sezione Obbligatoria

Contiene la logica eseguibile: statement SQL, assegnazioni, strutture di controllo. È il cuore del blocco.

- Statement SQL (SELECT, INSERT, UPDATE, DELETE)
- Strutture di controllo (IF, LOOP, CASE)
- Chiamate a procedure e funzioni

EXCEPTION – Sezione Opzionale

Cattura e gestisce gli errori runtime. Senza questa sezione, un errore interrompe il blocco e restituisce un messaggio grezzo.

Primo Blocco PL/SQL

Ecco il più semplice blocco PL/SQL funzionante. Dichiarazione di una variabile, assegnazione di un valore e chiusura del blocco. Copia questo codice in SQL Developer e premi F5 per eseguirlo.

```
DECLARE
  v_num NUMBER;          -- Dichiarazione di una variabile numerica
BEGIN
  v_num := 10;           -- Assegnazione del valore 10
  DBMS_OUTPUT.PUT_LINE('Il valore è: ' || v_num);
END;
/
```

Analisi del Codice

- `DECLARE` – inizia la sezione di dichiarazione
- `v_num NUMBER;` – variabile di tipo numerico
- `BEGIN` – inizia il corpo eseguibile
- `:=` – operatore di assegnazione (non =)
- `DBMS_OUTPUT.PUT_LINE` – stampa l'output
- `/` – segnala la fine del blocco allo strumento

Abilitare l'Output

Per vedere il risultato di `DBMS_OUTPUT`, devi abilitare il buffer di output nel tuo client:

```
SET SERVEROUTPUT ON
```

In **SQL Developer**: View → Dbms Output → clicca sul "+" verde. Senza questa impostazione, il messaggio non verrà visualizzato anche se il blocco viene eseguito correttamente.

✓ Output atteso: **Il valore è: 10**

Variabili in PL/SQL

Le variabili sono contenitori che memorizzano valori durante l'esecuzione del blocco. Ogni variabile deve essere dichiarata nella sezione `DECLARE` prima di essere utilizzata.

Dichiarazione

Sintassi: `nome_variabile tipo [:= valore_iniziale];`

```
v_nome    VARCHAR2(50) := 'Mario';
v_eta     NUMBER       := 30;
v_data    DATE         := SYSDATE;
v_flag    BOOLEAN      := TRUE;
```

Il nome deve iniziare con una lettera. Per convenzione si usa il prefisso `v_` per le variabili locali.

Assegnazione

Si usa l'operatore `:=` (non il semplice `=` come in SQL).

```
v_salario := 50000;
v_nome    := 'Rossi';
v_totale  := v_prezzo * v_qta;
```

È possibile assegnare il risultato di un'espressione o di una funzione. L'assegnazione avviene nella sezione `BEGIN`.

Scope (Visibilità)

Una variabile è visibile solo all'interno del blocco in cui è dichiarata. I blocchi nidificati possono vedere le variabili del blocco esterno, ma non viceversa.

```
DECLARE
  v_globale NUMBER := 100;
BEGIN
  DECLARE
    v_locale NUMBER := 50;
  BEGIN
    v_globale := v_globale +
    v_locale; -- OK
  END;
  -- v_locale non è visibile qui
END;
```

Tipi di Dato e Attributi

PL/SQL supporta i tipi di dato di Oracle e introduce attributi speciali che rendono il codice più robusto e mantenibile legandolo direttamente alla struttura del database.

Tipi di Dato Principali

NUMBER

Numeri interi e decimali. Supporta precisione e scala: `NUMBER(10,2)`

VARCHAR2

Stringhe di lunghezza variabile. Specificare sempre la dimensione: `VARCHAR2(100)`

DATE

Date e orari. Include anno, mese, giorno, ora, minuti e secondi.

BOOLEAN

Solo in PL/SQL (non in SQL). Valori: `TRUE`, `FALSE`, `NULL`

Attributi %TYPE e %ROWTYPE

Gli attributi ancorano il tipo della variabile alla definizione di una colonna o tabella nel database.

%TYPE – Il tipo segue la colonna:

```
v_salario employees.salary%TYPE;  
v_nome     employees.first_name%TYPE;
```

Se la colonna cambia tipo nel database, il codice PL/SQL si adatta automaticamente senza modifiche.

%ROWTYPE – Una variabile che rappresenta un'intera riga:

```
r_dipendente employees%ROWTYPE;  
-- Contiene tutti i campi della tabella
```

📌 Best practice: usa sempre `%TYPE` invece di hardcodare i tipi.

SELECT INTO: Recuperare Dati dal Database

Lo statement `SELECT INTO` è il modo più diretto per recuperare dati da una tabella Oracle all'interno di un blocco PL/SQL. I valori restituiti vengono assegnati a variabili dichiarate nella sezione `DECLARE`.

Sintassi Base

```
SELECT colonna1, colonna2
INTO   variabile1, variabile2
FROM   tabella
WHERE  condizione;
```

Regole Fondamentali

- Il `SELECT INTO` deve restituire **esattamente una riga**
- Se restituisce 0 righe → eccezione `NO_DATA_FOUND`
- Se restituisce più righe → eccezione `TOO_MANY_ROWS`
- Le variabili devono essere compatibili con i tipi delle colonne
- Usa `%TYPE` per garantire la compatibilità

Gestione delle Eccezioni

È buona norma gestire sempre le eccezioni legate al `SELECT INTO`:

```
DECLARE
    v_salario employees.salary%TYPE;
BEGIN
    SELECT salary
    INTO   v_salario
    FROM   employees
    WHERE  employee_id = 101;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Dipendente non trovato');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Più righe restituite');
END;
/
```

 Per query multi-riga si usano i **cursori espliciti** o i cicli `FOR`, argomento delle lezioni successive.

Esempio Completo: SELECT INTO con %TYPE

Mettiamo insieme tutti i concetti visti: dichiarazione con `%TYPE`, `SELECT INTO`, output con `DBMS_OUTPUT` e gestione delle eccezioni. Questo è il blocco PL/SQL completo che ogni sviluppatore dovrebbe saper scrivere.

```
SET SERVEROUTPUT ON

DECLARE
  -- Variabili ancorate alle colonne della tabella
  v_nome      employees.first_name%TYPE;
  v_cognome    employees.last_name%TYPE;
  v_salario    employees.salary%TYPE;
  v_dip        employees.department_id%TYPE;
BEGIN
  -- Recupero i dati del dipendente con ID 101
  SELECT first_name,
         last_name,
         salary,
         department_id
  INTO   v_nome,
         v_cognome,
         v_salario,
         v_dip
  FROM   employees
  WHERE  employee_id = 101;

  -- Output formattato
  DBMS_OUTPUT.PUT_LINE('--- Dati Dipendente ---');
  DBMS_OUTPUT.PUT_LINE('Nome:      ' || v_nome || ' ' || v_cognome);
  DBMS_OUTPUT.PUT_LINE('Salario:  € ' || v_salario);
  DBMS_OUTPUT.PUT_LINE('Dipartim.: ' || v_dip);

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Errore: dipendente non trovato.');
```

```
  WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('Errore: query restituisce più righe.');
```

```
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Errore generico: ' || SQLERRM);
END;
/
```

✓ Pratica Consigliata

Usa sempre `%TYPE` per le variabili. Il codice si adatta automaticamente se la struttura della tabella cambia.

⚠ Attenzione

Il `SELECT INTO` funziona solo per query che restituiscono una singola riga. Per più righe, usa i cursori.

🔍 Prossimi Passi

Nella prossima lezione: strutture di controllo `IF/THEN`, cicli `LOOP` e introduzione ai cursori espliciti.